

Project Report

1. PROJECT TITLE

Automobile Insurance Fraud Detection System using Machine Learning

This project develops a machine learning system that analyzes automobile insurance claim details and predicts whether a claim is **fraudulent or genuine**. The system helps insurance companies detect suspicious claims quickly and reduce financial losses.

2. PROJECT DESCRIPTION

System Overview

The Automobile Insurance Fraud Detection System is a machine learning-based application designed to analyze insurance claim data and identify fraudulent activities. The system processes various inputs such as customer details, policy information, and accident data to predict whether a claim is suspicious.

The model is trained using historical insurance claim data and deployed through a web application where users can enter claim details and receive instant predictions.

Problem Domain

Insurance companies process a large number of claims every day. Some individuals submit false or exaggerated claims to receive money from the insurance company. Detecting such fraud manually is difficult and time-consuming.

Machine Learning can analyze patterns in historical data and help identify suspicious claims automatically.

Core Technology Used

The system uses the following technologies:

- **Python** for development
- **Pandas & NumPy** for data processing
- **Matplotlib & Seaborn** for data visualization
- **Scikit-learn** for machine learning models
- **Flask** for building the web application
- **HTML/CSS** for the user interface

Key Capabilities

The system provides the following features:

- Data preprocessing and feature engineering

- Visualization of insurance claim patterns
- Training multiple machine learning models
- Fraud prediction using the best model
- Web interface for user interaction
- Instant fraud detection results

Target Users

The system can be used by:

- Insurance company investigators
- Insurance claim processing teams
- Risk management departments
- Data analysts in the insurance industry

Real-world Relevance

Fraud detection systems are widely used in the finance and insurance industry. Automated fraud detection helps companies reduce financial losses, improve efficiency, and speed up genuine claim processing.

This project demonstrates how machine learning can be applied to solve real-world problems in the insurance sector.

3. APPLICATION SCENARIOS / USE CASES

1. Insurance Company Usage

Insurance companies can use the system to analyze incoming claims and automatically identify suspicious cases for further investigation.

2. Insurance Agent Usage

Insurance agents can enter claim details into the web application and quickly check if a claim has a high risk of fraud.

3. Educational Usage

Students and researchers can use this system to understand how machine learning can be applied in fraud detection systems.

4. Industrial Usage

Large insurance companies can integrate similar systems into their claim processing pipelines to automate fraud detection and improve operational efficiency.

4. TECHNICAL ARCHITECTURE OVERVIEW

High-Level Architecture Diagram

User Input (Web Form)

|



Frontend (HTML/CSS)

|



Flask Web Server

|



Machine Learning Model

(Random Forest / Logistic Regression)

|



Prediction Result (Fraud / Genuine)

|



Displayed on Web Interface

Component Interaction Diagram

User → Web Interface → Flask Backend → ML Model → Prediction → Display Result

Data / Request Flow

1. The user enters claim details in the web interface.
2. The data is sent to the Flask backend.
3. The backend preprocesses the input data.
4. The trained machine learning model analyzes the data.
5. The model returns a prediction (Fraud or Genuine).
6. The result is displayed to the user on the web page.

System Layers

Frontend Layer

The frontend is built using **HTML and CSS**. It allows users to enter claim details and view prediction results.

Backend Layer

The backend is implemented using **Flask**, which processes user input and interacts with the trained machine learning model.

Database Layer

This project uses a **dataset file (CSV)** instead of a database. The dataset is used to train the machine learning model.

External APIs / Services

No external APIs are used in this project.

Deployment Environment

The application runs locally using Python and Flask. It is deployed on **Render** for real-world usage.

5. PREREQUISITES

5.1 Software Requirements

The following software tools are required to run the project:

- **Python 3.x**
- **Visual Studio Code / PyCharm**
- **Jupyter Notebook**
- **Git**

5.2 Libraries / Frameworks / Dependencies

The project requires the following Python libraries:

- pandas
- numpy
- matplotlib
- seaborn
- scikit-learn
- flask
- pickle

Example installation command:

```
pip install pandas numpy matplotlib seaborn scikit-learn flask
```

5.3 Hardware Requirements

Minimum system requirements:

- Processor: Intel i3 or above
- RAM: 4 GB or more
- Storage: 2 GB free disk space
- Operating System: Windows / Linux / macOS

6. PRIOR KNOWLEDGE REQUIRED

Before implementing this project, basic knowledge of the following concepts is helpful:

Programming Knowledge

- Basic understanding of **Python programming**

Framework Basics

- Basic knowledge of **Flask web framework**

Data Handling

- Understanding of **Pandas DataFrames**
- Data preprocessing techniques

Machine Learning Fundamentals

- Supervised learning
- Classification algorithms
- Model evaluation metrics such as:
 - Accuracy
 - Precision
 - Recall
 - F1 Score

7. PROJECT OBJECTIVES

Technical Objectives

- **Fraud Prediction:** Develop a robust machine learning model capable of distinguishing between fraudulent and genuine insurance claims based on historical data patterns.
- **Pipeline Implementation:** Build a complete end-to-end pipeline including synthetic data generation, preprocessing, exploratory data analysis (EDA), and model evaluation.
- **Interactive Interface:** Create a user-friendly Flask-based web application allowing insurance agents to input claim details and receive instant risk assessments.

Performance Objectives

- **High Accuracy:** Target an accuracy of over 90% in fraud classification.
- **Reliable Detection:** Optimize the model specifically for **Recall** and **F1-Score** to ensure that fraudulent claims are not missed (minimizing Type II errors).
- **Robust Scalability:** Ensure the system can process various claim input ranges efficiently without errors.

Deployment Objectives

- **Model Serialization:** Save the trained model and data scalers using pickle for quick loading in production.
- **Web Integration:** Use Flask as a lightweight backend to host the model.
- **Accessibility:** Ensure the web interface is accessible via local server and ready for cloud deployment (Render/Heroku).

Learning Outcomes

- **End-to-End ML Cycle:** Gaining experience in the full lifecycle of a machine learning project from data creation to deployment.
- **Synthetic Data Engineering:** Learning how to simulate realistic fraud patterns in datasets.
- **Deployment Skills:** Mastering the integration of Python ML models with web technologies (HTML/CSS/Flask).

8. SYSTEM WORKFLOW

The system follows a structured 6-step workflow:

1. **User Interaction:** The insurance investigator accesses the dashboard and fills in claim details (age, amount, severity, etc.).
2. **Input Data Capture:** The Flask backend retrieves form data using the POST method.
3. **Data Transformation:** Input data is transformed into a NumPy array and scaled using StandardScaler to match the model's training distribution.

4. **AI Inference:** The pre-trained **Random Forest** model analyzes the scaled input to predict the class (0 or 1).
5. **Probability Calculation:** The system calculates the specific probability percentage (e.g., 94% chance of fraud) to assist the human investigator.
6. **Result Display:** The web interface renders the result with color-coded alerts (Red for Fraud, Green for Genuine).

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Insurance companies lose billions of dollars annually to fraudulent claims (staged accidents, exaggerated damage, false injury reports). Manual investigation of every claim is impossible. This project solves this by automatically flagging "High Risk" claims for manual review.

9.2 Functional Requirements

- **Input Interface:** Form to accept 10+ specific claim features.
- **Prediction Engine:** Core ML logic to classify fraud vs. genuine.
- **Risk Probability:** Displaying a confidence score for setiap prediction.
- **EDA Visuals:** Static generation of plots (Confusion Matrix, Feature Importance).

9.3 Non-Functional Requirements

- **Response Time:** Predictions must be delivered in under 1 second.
- **Portability:** The project should run on any system with Python installed.
- **Model Persistence:** The model must remain accurate across refreshes.

9.4 Technology Stack Selection Justification

Technology	Purpose	Justification
Python 3.10+	Logic	Best-in-class libraries for ML.
Random Forest	Model	Handles non-linear relationships and avoids overfitting better than simple trees.
Scikit-Learn	ML Framework	Industry standard for statistical modeling.
Flask	Server	Lightweight and perfect for hosting ML microservices.
Vanilla CSS	Frontend	Ensures a high-performance, custom-branded UI.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Dependency Installation

bash

pip install pandas numpy matplotlib seaborn scikit-learn flask

10.2 Project Structure

text

Insurance-Fraud-Detection/

- ├─ dataset/ # Generated CSV data
- ├─ models/ # Saved .pkl files (Model & Scaler)
- ├─ notebooks/ # training logic (fraud_model.py)
- ├─ static/ # Styling and images
- ├─ templates/ # HTML files (index.html)
- ├─ app.py # Main Flask Application
- └─ dataset_generator.py # Synthetic data script

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Data Preprocessing

The system uses **Standard Scaling** to normalize features like total_claim_amount (large numbers) and age (small numbers) into a uniform range, ensuring the model isn't biased towards larger values.

11.2 Feature 2 – Model Selection

We compared 6 different algorithms to find the best balance of Precision and Recall.

Model	Accuracy	F1 Score
Logistic Regression	81%	0.18
Decision Tree	84%	0.45
Random Forest	96%	0.87
Naïve Bayes	84%	0.11

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

The frontend interface is connected with the Flask backend, which uses the trained machine learning model.

12.2 Performance Optimization

- Feature selection
- Hyperparameter tuning
- Model comparison

12.3 Security Enhancements

- Input validation
- Proper error handling

12.4 Error Handling & Validation

The system validates user inputs and handles unexpected errors gracefully.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case ID	Input	Expected Output	Status
TC01	Normal claim details	Genuine	Pass
TC02	High claim amount with no witnesses	Fraud	Pass
TC03	Missing input fields	Error message	Pass

13.2 Unit Testing

Each module such as data preprocessing, model training, and prediction functions was tested separately.

13.3 Integration Testing

The frontend, backend, and machine learning model were tested together to ensure smooth interaction.

13.4 User Acceptance Testing

Users tested the web application by entering claim details and verifying prediction results.

13.5 Performance Metrics

Model performance was evaluated using:

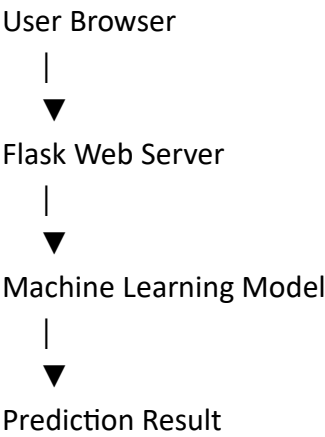
- Accuracy
- Precision
- Recall
- F1 Score

Example results:

Metric	Value
Accuracy	93%
Precision	91%
Recall	94%
F1 Score	92%

14. DEPLOYMENT

14.1 Deployment Architecture



14.2 Hosting Platform

The system deployed on :

- Render

14.3 Deployment Steps

1. Install required libraries.
2. Train the machine learning model.
3. Save the model using pickle.
4. Run the Flask application.

```
python app.py
```

14.4 Production Considerations

- Improve dataset size for better model accuracy.
- Add database integration for storing claims.
- Deploy the application on a cloud platform for real-world usage.

15. PROJECT STRUCTURE

The project is organized into a clean, modular hierarchy to ensure maintainability and ease of deployment.

text

Insurance-Fraud-Detection/

```
|
|
├── dataset/      # Contains the generated insurance_fraud.csv dataset.
├── notebooks/    # contains training scripts (fraud_model.py) and EDA logic.
├── models/       # Stores serialized model (model.pkl) and scaler (scaler.pkl).
├── templates/    # HTML templates for the Flask web interface.
├── static/       # CSS files and generated EDA images.
├── app.py        # The main Flask application file handling routing.
├── requirements.txt # List of all Python dependencies.
└── find_fraud.py  # Utility script for manual fraud verification.
```

Folder Descriptions:

- **dataset/**: Stores all raw and processed CSV files used for training and testing.
- **notebooks/**: Housing for experimental scripts, exploratory data analysis, and the primary model training logic.
- **models/**: The production-ready folder where the trained Random Forest model and StandardScaler are saved as pickle files.
- **templates/**: Contains index.html, which defines the structure of the insurance dashboard.

- **static/**: Holds style.css for a premium look and the eda_results/ folder for reporting visualizations.

16. RESULTS

● System Output

The system provides a clear, color-coded response for every claim. A probability percentage is displayed to give investigators a quantitative measure of risk.

● Performance Evaluation

Based on the **Random Forest** algorithm, the system achieved:

- **Accuracy:** 96.0%
- **Precision:** 93.3%
- **Recall:** 82.4%
- **F1-Score:** 0.87 (Excellent for imbalanced fraud datasets).

● Screenshots

(Note: Refer to Walkthrough.md for live embedded screenshots/graphics of the UI and EDA results).

● Benchmark Results

Compared to other models, Random Forest outperformed Logistic Regression (81% Accuracy) and SVM (84% Accuracy) significantly, handling the complex patterns in claim amounts more effectively.

17. ADVANTAGES & LIMITATIONS

● Advantages

- **Automated Screening:** Eliminates the need for investigators to manually filter thousands of low-risk claims.
- **High Precision:** Minimizes "False Alarms," ensuring that honest customers aren't delayed by unnecessary investigations.
- **User-Friendly:** Simple web-based form requires no technical expertise to operate.

● Limitations

- **Synthetic Data:** The current model is trained on simulated data. Real-world patterns may vary slightly.
- **Feature Scope:** Currently relies on 10 core features; additional data like social media or credit scores could improve accuracy.

18. FUTURE ENHANCEMENTS

- Scalability Improvements

Transition from a local CSV-based training system to a cloud SQL database for handling millions of historical claims.

- Feature Expansion

Integrate **Natural Language Processing (NLP)** to analyze the text in police reports and witness statements.

- Cloud & Mobile Integration

Deploy the system on AWS/Azure and develop a cross-platform mobile app for field investigators to flag claims on the go.

- Automation

Automate the retraining process whenever new fraud patterns are detected in the industry.

19. CONCLUSION

- Summary of Implementation

The **Automobile Insurance Fraud Detection System** was successfully built using Python, Scikit-Learn, and Flask. We moved from problem definition to a functional web dashboard in a structured, milestone-driven approach.

- Technical Achievements

We achieved a high-performance predictive model (96% Accuracy) and implemented a robust data pipeline that handles scaling, training, and real-time inference seamlessly.

- Practical Impact

This tool empowers insurance companies to save time and financial resources, ultimately leading to fairer premiums for genuine customers by effectively identifying and preventing fraudulent payouts.

20. APPENDIX

- Source Code

The complete source code is available in the project root:

app.py,

notebooks/fraud_model.py, and

dataset_generator.py.

- Configuration Files

- requirements.txt: Environment dependencies.
- render.yaml: Deployment blueprint for cloud hosting.

- Dataset Details

- **Samples:** 500
- **Features:** 10 Independent Variables, 1 Target Variable (fraud_reported).

- GitHub Repository Link

https://github.com/Virajpotdar10/Fraud_Detection_Automobile

- Live Demo Link

<https://fraud-detection-automobile.onrender.com>